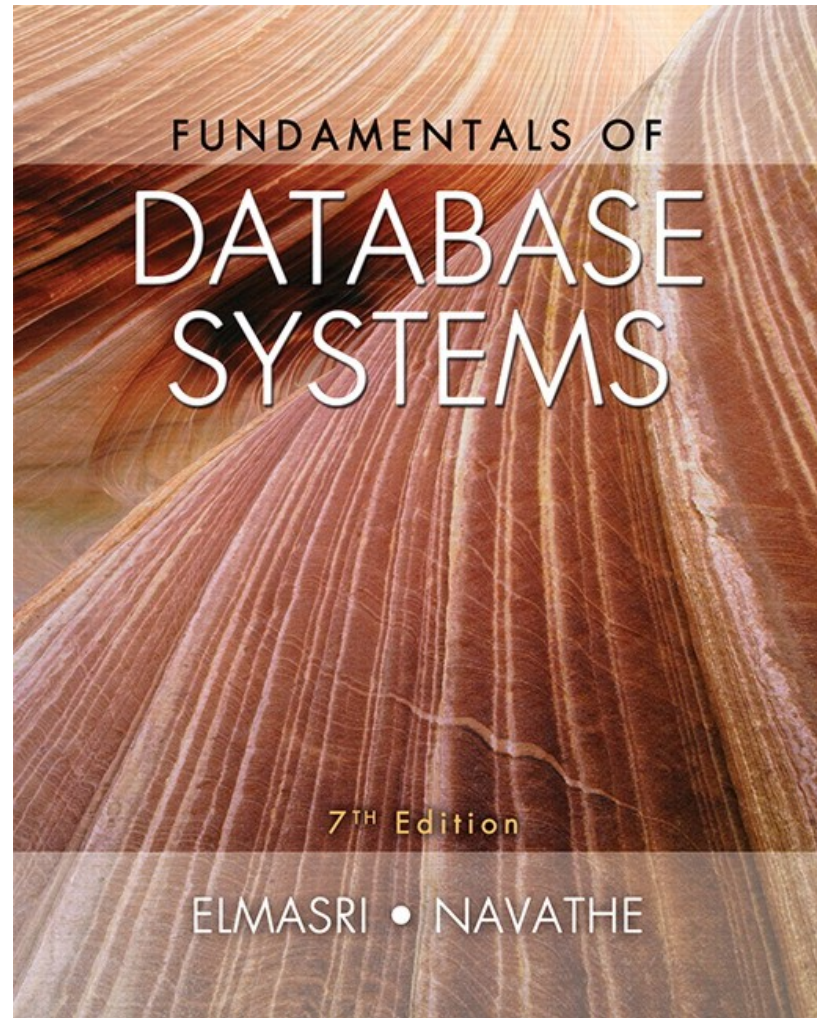




CHAPTER 4

Enhanced Entity-Relationship (EER) Modeling

Chapter Outline

- EER stands for Enhanced ER or Extended ER
- EER Model Concepts
 - Includes all modeling concepts of basic ER
 - Additional concepts:
 - subclasses/superclasses
 - specialization/generalization
 - categories (UNION types)
 - attribute and relationship inheritance
 - Constraints on Specialization/Generalization

SUBCLASSES, SUPERCLASSES, AND INHERITANCE

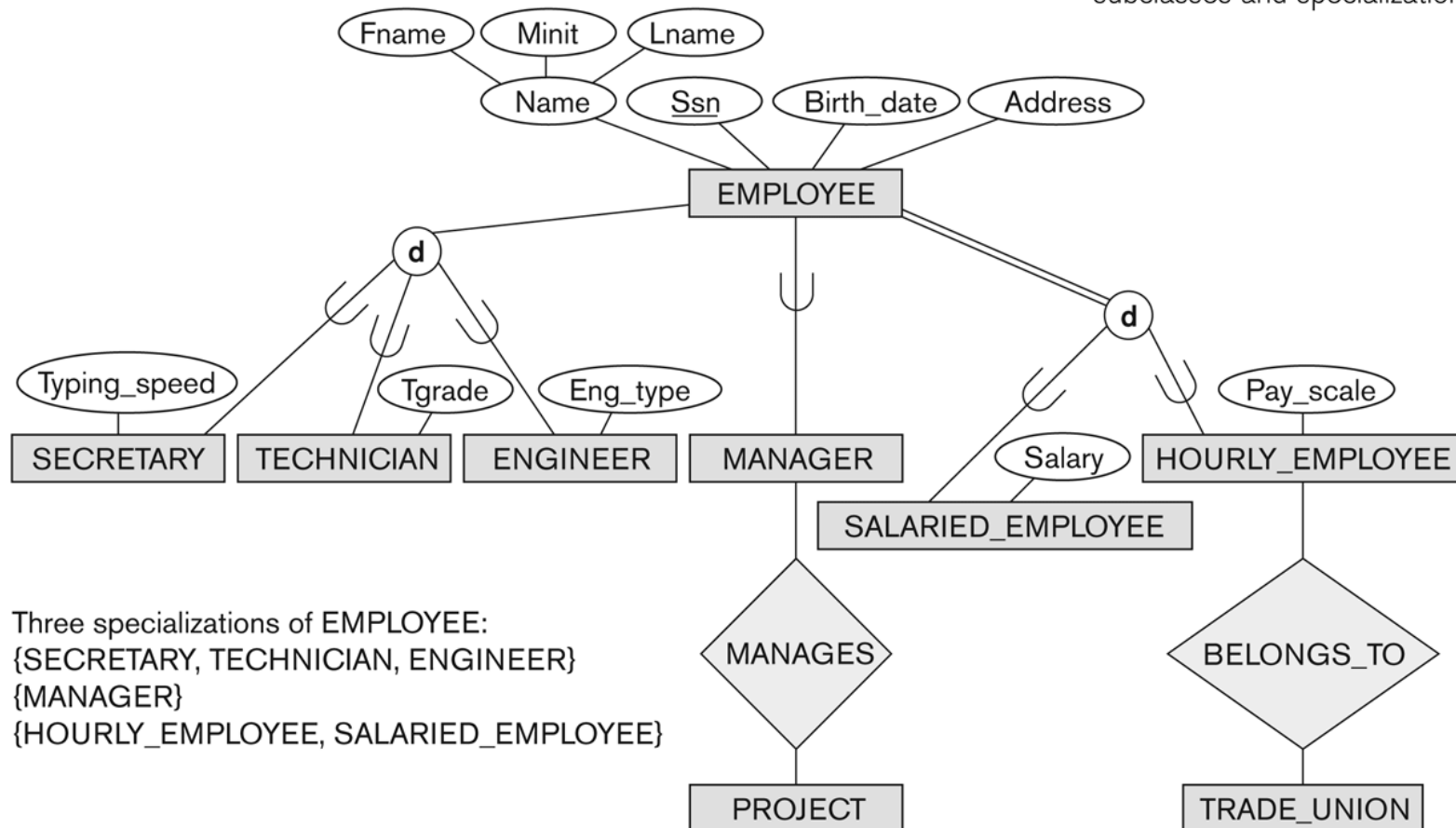
Subclasses and Superclasses (1)

- An entity type may have additional meaningful subgroupings of its entities
 - Example: EMPLOYEE may be further grouped into:
 - SECRETARY, ENGINEER, TECHNICIAN, ...
 - Based on the EMPLOYEE's Job
 - MANAGER
 - EMPLOYEEs who are managers (the role they play)
 - SALARIED_EMPLOYEE, HOURLY_EMPLOYEE
 - Based on the EMPLOYEE's method of pay
- EER diagrams extend ER diagrams to represent these additional subgroupings, called *subclasses* or *subtypes*

Subclasses and Superclasses

Figure 4.1

EER diagram notation to represent subclasses and specialization.



Subclasses and Superclasses (2)

- Each of these subgroupings is a subset of EMPLOYEE entities
- Each is called a subclass of EMPLOYEE
- EMPLOYEE is the superclass or supertype for each of these subclasses
- These are called **superclass/subclass** relationships:
 - EMPLOYEE/SECRETARY
 - EMPLOYEE/TECHNICIAN
 - EMPLOYEE/MANAGER
 - ...

Subclasses and Superclasses (3)

- These are also called IS-A relationships
 - SECRETARY IS-A EMPLOYEE, TECHNICIAN IS-A EMPLOYEE,
- Note: An entity that is member of a subclass represents the same real-world entity as some member of the superclass:
 - The subclass member is the same entity in a *distinct specific role*
 - An entity cannot exist in the database merely by being a member of a subclass; it must also be a member of the superclass
 - A member of the superclass can be optionally included as a member of any number of its subclasses

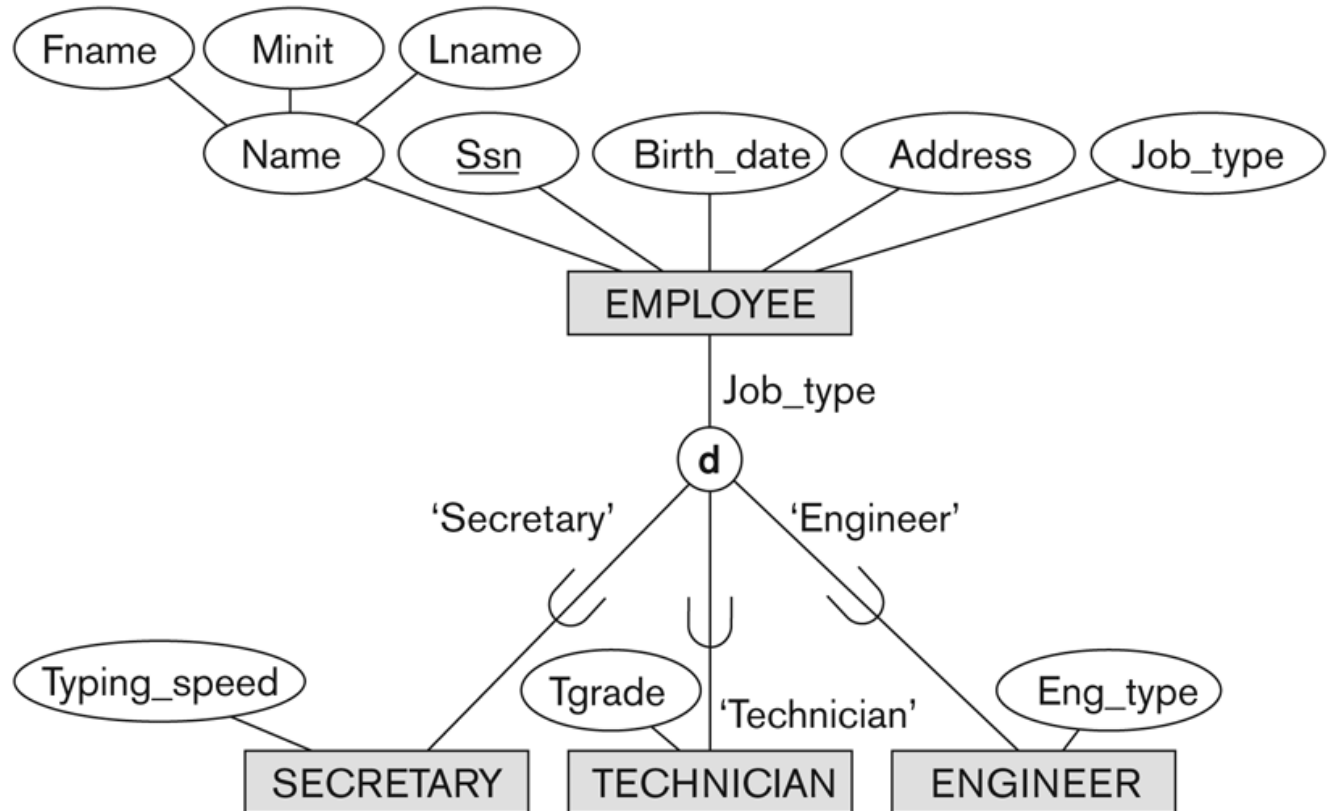
Subclasses and Superclasses (4)

- Examples:
 - A salaried employee who is also an engineer belongs to the two subclasses:
 - ENGINEER, and
 - SALARIED_EMPLOYEE
 - A salaried employee who is also an engineering manager belongs to the three subclasses:
 - MANAGER,
 - ENGINEER, and
 - SALARIED_EMPLOYEE
- It is not necessary that every entity in a superclass be a member of some subclass

Representing Specialization in EER Diagrams

Figure 4.4

EER diagram notation for an attribute-defined specialization on Job_type.



Attribute Inheritance in Superclass / Subclass Relationships

- An entity that is member of a subclass *inherits*
 - All attributes of the entity as a member of the superclass
 - All relationships of the entity as a member of the superclass
- Example:
 - In the previous slide, SECRETARY (as well as TECHNICIAN and ENGINEER) inherit the attributes Name, SSN, ..., from EMPLOYEE
 - Every SECRETARY entity will have values for the inherited attributes

SPECIALIZATION AND GENERALIZATION

Specialization (1)

- Specialization is the **process** of defining a set of subclasses of a superclass
- The set of subclasses is based upon some distinguishing characteristics of the entities in the superclass
 - Example: {SECRETARY, ENGINEER, TECHNICIAN} is a specialization of EMPLOYEE based upon *job type*.
 - Example: *MANAGER is a specialization of EMPLOYEE based on the role the employee plays*
 - May have several specializations of the same superclass

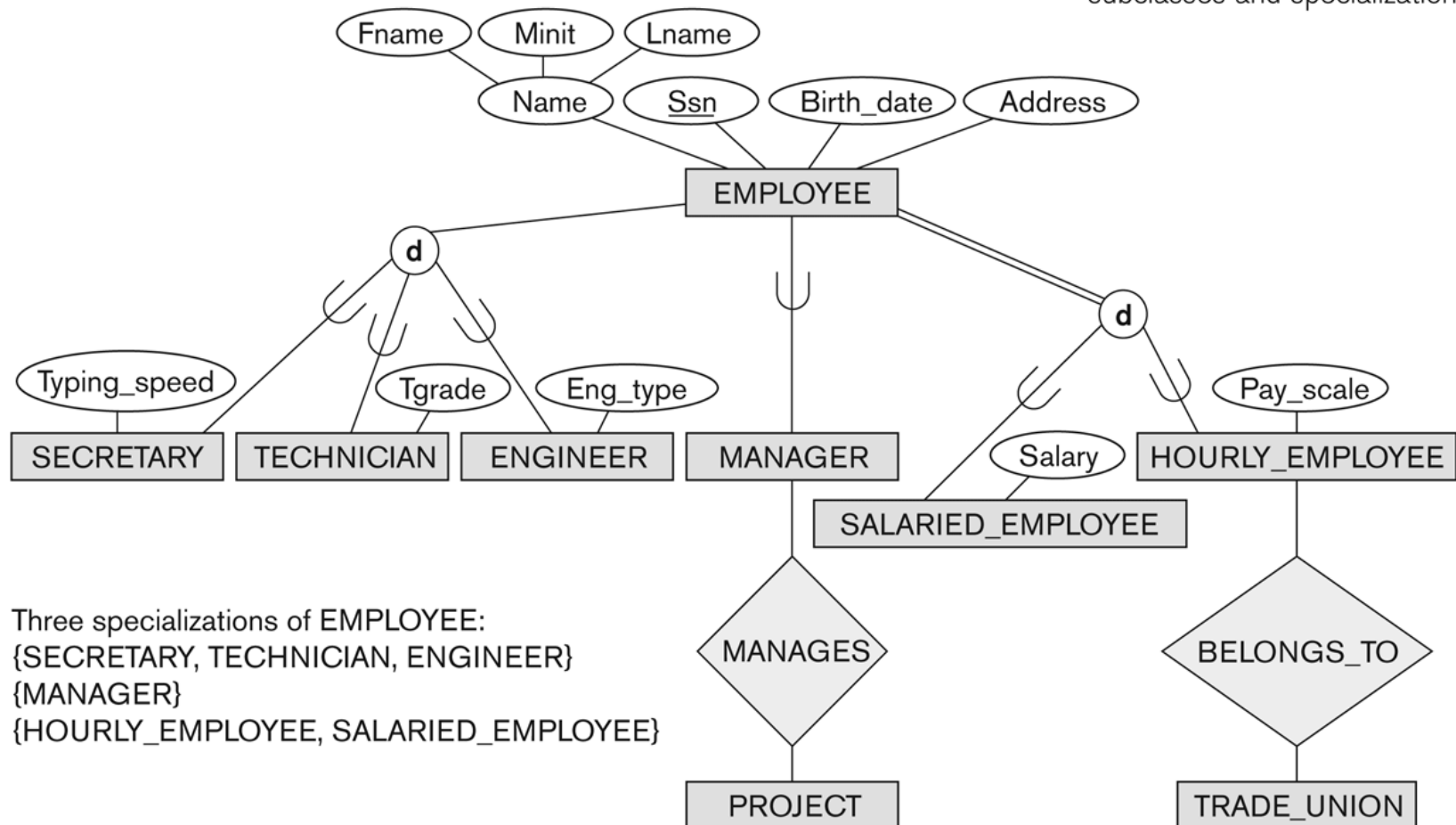
Specialization (2)

- Example: Another specialization of EMPLOYEE based on *method of pay* is {SALARIED_EMPLOYEE, HOURLY_EMPLOYEE}.
 - Superclass/subclass relationships and specialization can be diagrammatically represented in EER diagrams
 - Attributes of a subclass are called *specific* or *local* attributes.
 - For example, the attribute TypingSpeed of SECRETARY
 - The subclass can also participate in specific relationship types.
 - For example, a relationship BELONGS_TO of HOURLY_EMPLOYEE

Specialization (3)

Figure 4.1

EER diagram notation to represent subclasses and specialization.



Generalization

- Generalization is the reverse of the specialization process
- Several classes with common features are generalized into a superclass;
 - original classes become its subclasses
- Example: CAR, TRUCK generalized into VEHICLE;
 - both CAR, TRUCK become subclasses of the superclass VEHICLE.
 - We can view {CAR, TRUCK} as a specialization of VEHICLE
 - Alternatively, we can view VEHICLE as a generalization of CAR and TRUCK

Generalization (2)

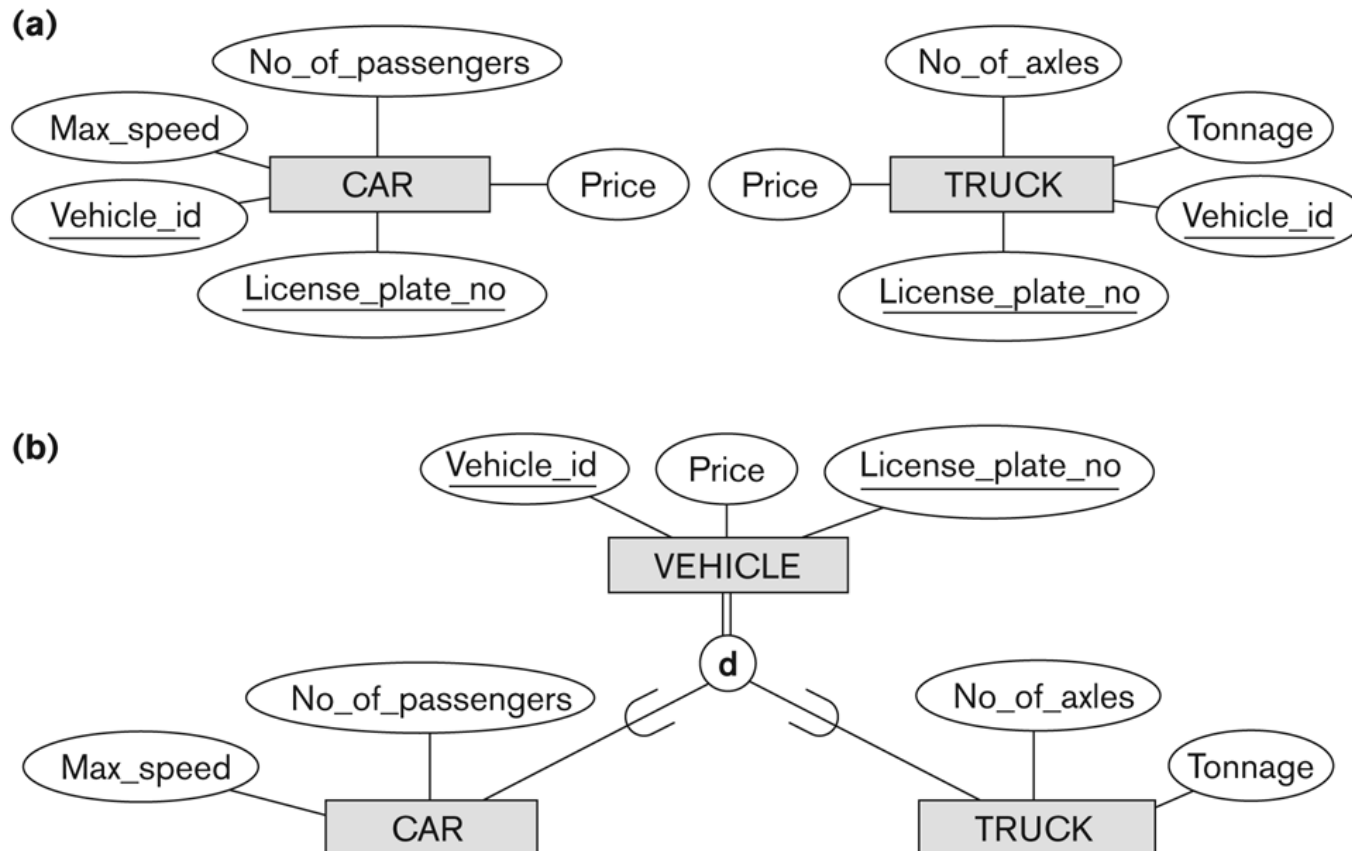


Figure 4.3
Generalization. (a) Two entity types, CAR and TRUCK.
(b) Generalizing CAR and TRUCK into the superclass VEHICLE.

Generalization and Specialization (1)

- Diagrammatic notations are sometimes used to distinguish between generalization and specialization
 - Arrow pointing to the generalized superclass represents a generalization
 - Arrows pointing to the specialized subclasses represent a specialization
 - We *do not use* this notation because it is often subjective as to which process is more appropriate for a particular situation
 - We advocate not drawing any arrows

Generalization and Specialization (2)

- Data Modeling with Specialization and Generalization
 - A superclass or subclass represents a collection (or set or grouping) of entities
 - It also represents a particular *type of entity*
 - Shown in rectangles in EER diagrams (as are entity types)
 - We can call all entity types (and their corresponding collections) **classes**, whether they are entity types, superclasses, or subclasses

Constraints and Characteristics of Specialization And Generalization Hierarchies

Types of Specialization

- **Predicate-defined** (or condition-defined) : based on some predicate. E.g., based on value of an attribute, say, Job-type, or Age.
- **Attribute-defined**: shows the name of the attribute next to the line drawn from the superclass toward the subclasses (see Fig. 4.1)
- **User-defined**: membership is defined by the user on an entity by entity basis

Types of Specialization (Predicate-defined)

- If we can determine exactly those entities that will become members of each subclass by a condition, the subclasses are called predicate-defined (or condition-defined) subclasses
 - Condition is a constraint that determines subclass members
 - Display a predicate-defined subclass by writing the predicate condition next to the line attaching the subclass to its superclass

Types of Specialization (attribute-defined)

- If all subclasses in a specialization have membership condition on same attribute of the superclass, specialization is called an attribute-defined specialization
 - Attribute is called the defining attribute of the specialization
 - Example: JobType is the defining attribute of the specialization {SECRETARY, TECHNICIAN, ENGINEER} of EMPLOYEE

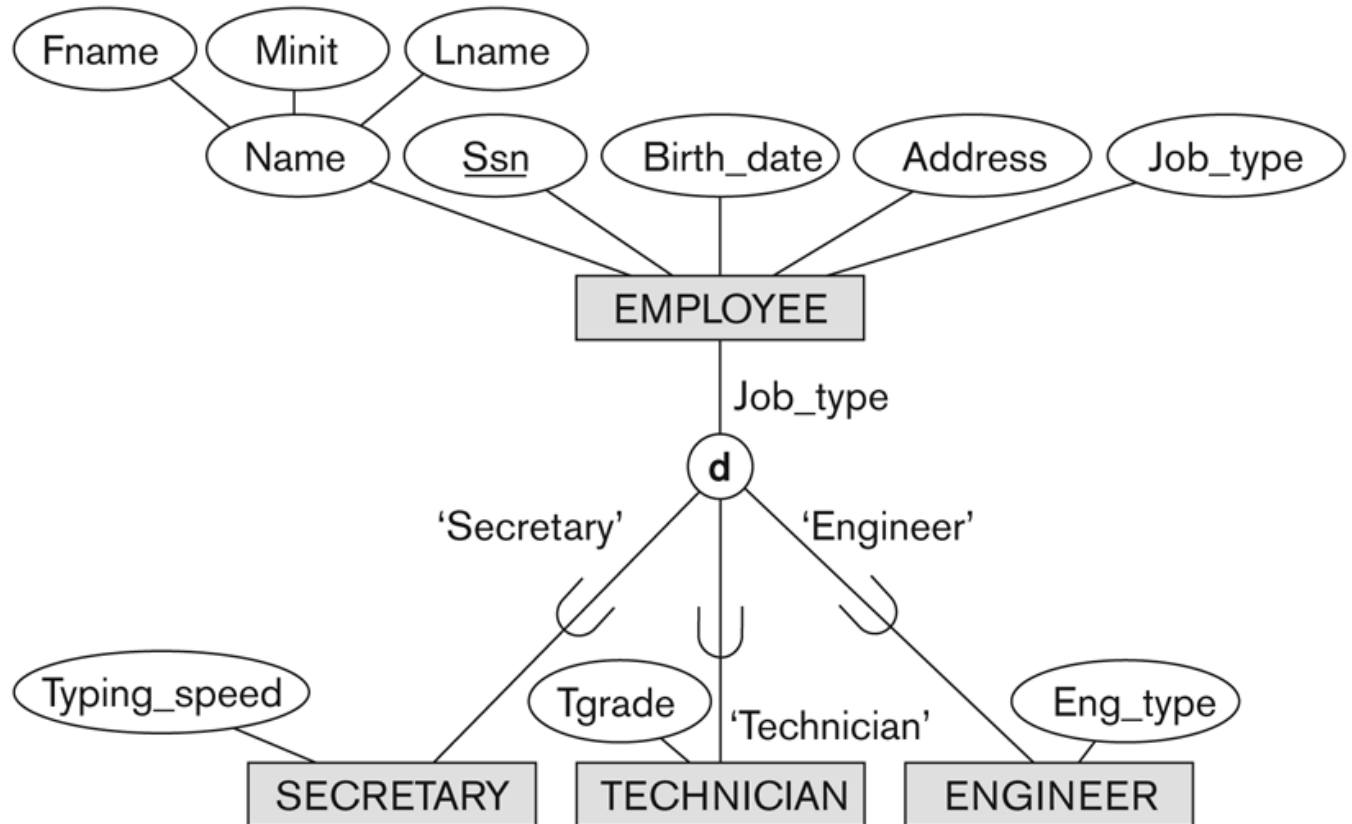
Types of Specialization (user-defined)

- If no condition determines membership, the subclass is called user-defined
 - Membership in a subclass is determined by the database users by applying an operation to add an entity to the subclass
 - Membership in the subclass is specified individually for each entity in the superclass by the user

Displaying an attribute-defined specialization in EER diagrams

Figure 4.4

EER diagram notation for an attribute-defined specialization on Job_type.



Constraints on Specialization and Generalization

- Two basic constraints can apply to a specialization/generalization:
 - Disjointness Constraint:
 - Completeness Constraint:

Constraints on Specialization and Generalization

- **Disjointness Constraint:**
 - Specifies that the subclasses of the specialization must be *disjoint*:
 - an entity can be a member of at most one of the subclasses of the specialization
 - Specified by **d** in EER diagram
 - If not disjoint, specialization is *overlapping*:
 - that is the same entity may be a member of more than one subclass of the specialization
 - Specified by **o** in EER diagram

Constraints on Specialization and Generalization

- **Completeness (Exhaustiveness) Constraint:**
 - *Total* specifies that every entity in the superclass must be a member of some subclass in the specialization/generalization
 - Shown in EER diagrams by a **double line**
 - *Partial* allows an entity not to belong to any of the subclasses
 - Shown in EER diagrams by a single line

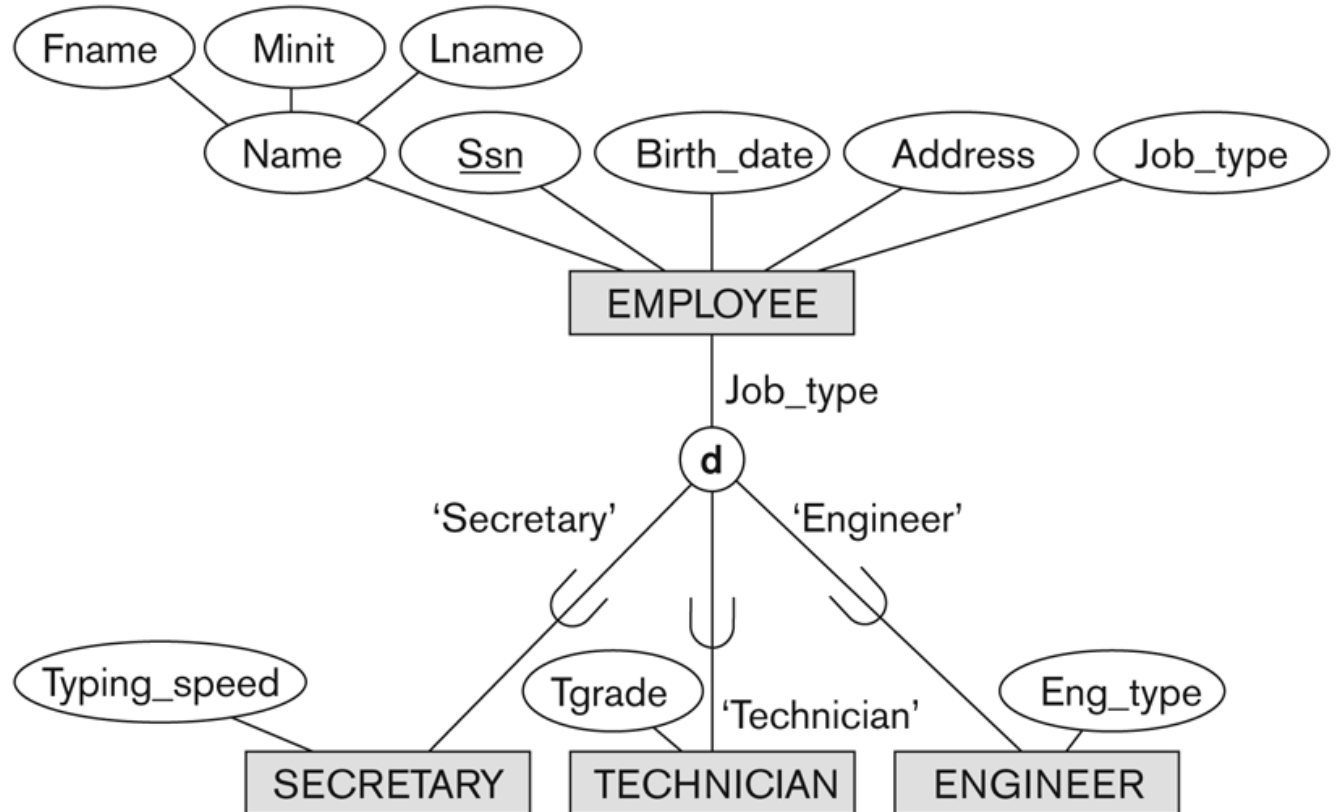
Constraints on Specialization and Generalization

- Hence, we have four types of specialization/generalization:
 - Disjoint, total
 - Disjoint, partial
 - Overlapping, total
 - Overlapping, partial
- **Note:** *Generalization usually is total because the superclass is derived from the subclasses.*

Example of disjoint partial Specialization

Figure 4.4

EER diagram notation for an attribute-defined specialization on Job_type.



Example of overlapping total Specialization

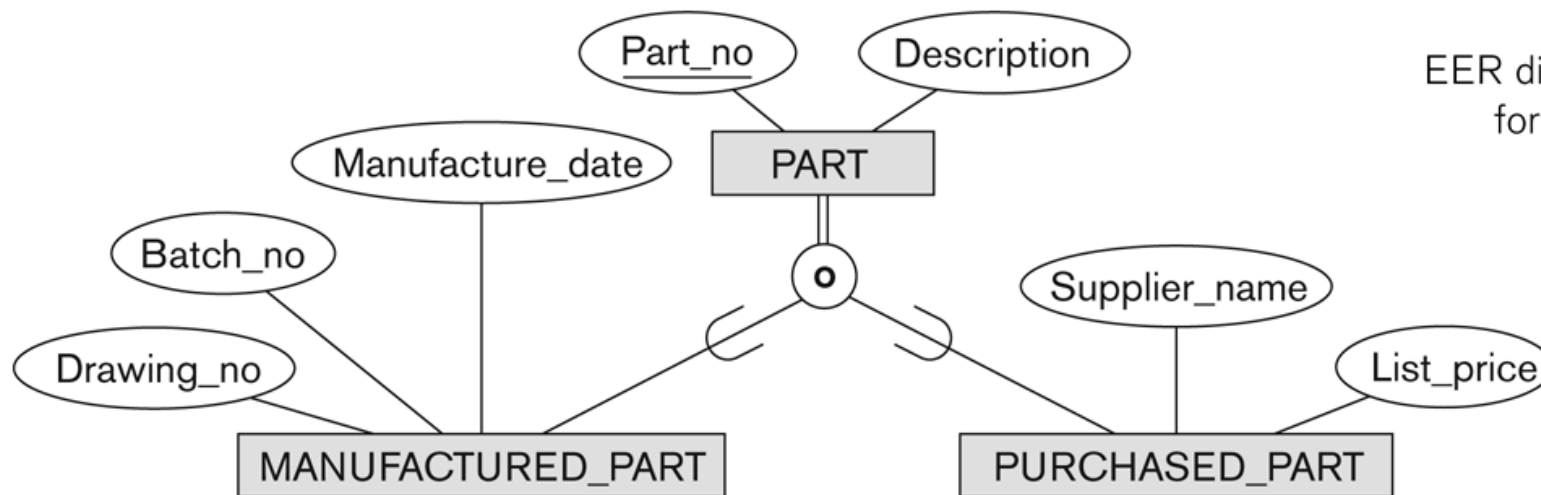


Figure 4.5
EER diagram notation
for an overlapping
(nondisjoint)
specialization.

Insertion and Deletion Rules

- Insertion and deletion rules apply to specialization (and generalization) as a consequence of the constraints specified earlier:
 - **Deleting** an entity from a superclass implies that it is automatically deleted from all the subclasses
 - **Inserting** an entity in a superclass implies that the entity is mandatorily inserted in all predicate-defined (or attribute-defined) subclasses for which the entity satisfies the defining predicate
 - **Inserting** an entity in a superclass of a total specialization implies that the entity is mandatorily inserted in at least one of the subclasses of the specialization

Specialization/Generalization Hierarchies, Lattices & Shared Subclasses (1)

- A subclass may itself have further subclasses specified on it
 - forms a hierarchy or a lattice
- **Hierarchy** has a constraint that every subclass has only one superclass (called **single inheritance**); this is basically a **tree structure**
- In a **lattice**, a subclass can be subclass of more than one superclass (called **multiple inheritance / shared class**)

Shared Subclass “Engineering_Manager”

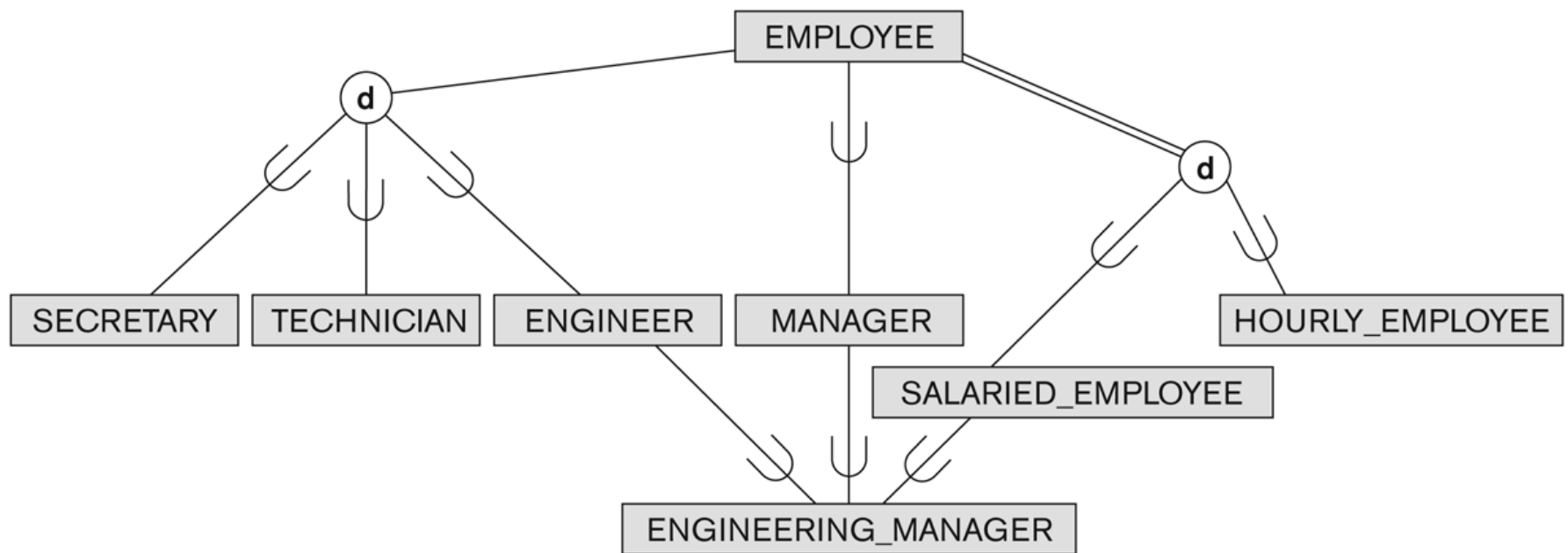


Figure 4.6

A specialization lattice with shared subclass ENGINEERING_MANAGER.

Specialization/Generalization Hierarchies, Lattices & Shared Subclasses (2)

- In a lattice or hierarchy, a subclass inherits attributes not only of its direct superclass, but also of all its predecessor superclasses
- We can have:
 - *specialization* hierarchies or lattices, or
 - *generalization* hierarchies or lattices,
 - depending on how they were *derived*
- We just use *specialization* (to stand for the end result of either specialization or generalization)

Specialization/Generalization Hierarchies, Lattices & Shared Subclasses (3)

- In *specialization*, start with an entity type and then define subclasses of the entity type by successive specialization
 - called a *top down* conceptual refinement process
- In *generalization*, start with many entity types and generalize those that have common properties
 - Called a *bottom up* conceptual synthesis process
- In practice, a *combination of both processes* is usually employed

Example: A UNIVERSITY DB

- The database keeps track of three types of persons: employees, alumni, and students.
 - A person can belong to one, two, or all three of these types.
 - Each person has a name, SSN, sex, address, and birth date.
- For each alumnus, a record of the (degree or degrees) that s/he earned at the university is kept, including the name of the degree, the year granted, and the major department
- Each student has a major department
- Every employee has a salary, and there are three types of employees: faculty, staff, and student assistants.
 - Each employee belongs to exactly **one** of these types

- Each faculty has a rank_
- ○ Each staff member has a staff position
- Student assistants are **classified** further as either research assistants or teaching assistants, and the percent of time that they work is recorded in the database.
 - Research assistants have their research project stored
 - Teaching assistants have the current course they work on
- Students are further classified as either graduate or undergraduate, with the specific attributes degree program for graduate students and class for undergraduates.

- The database keeps track of three types of persons: employees, alumni, and students.
 - A person can belong to one, two, or all three of these types.
 - Each person has a name, SSN, sex, address, and birth date.
- For each alumnus, a record of the (degree or degrees) that s/he earned at the university is kept, including the name of the degree, the year granted, and the major department
- Each student has a major department
- Every employee has a salary, and there are three types of employees: faculty, staff, and student assistants.
 - Each employee belongs to exactly one of these types
- Each faculty has a rank_
- ○ Each staff member has a staff position
- Student assistants are **classified** further as either research assistants or teaching assistants, and the percent of time that they work is recorded in the database.
 - Research assistants have their research project stored
 - Teaching assistants have the current course they work on
- Students are further classified as either graduate or undergraduate, with the specific attributes degree program for graduate students and class for undergraduates.

Specialization / Generalization Lattice Example (UNIVERSITY)

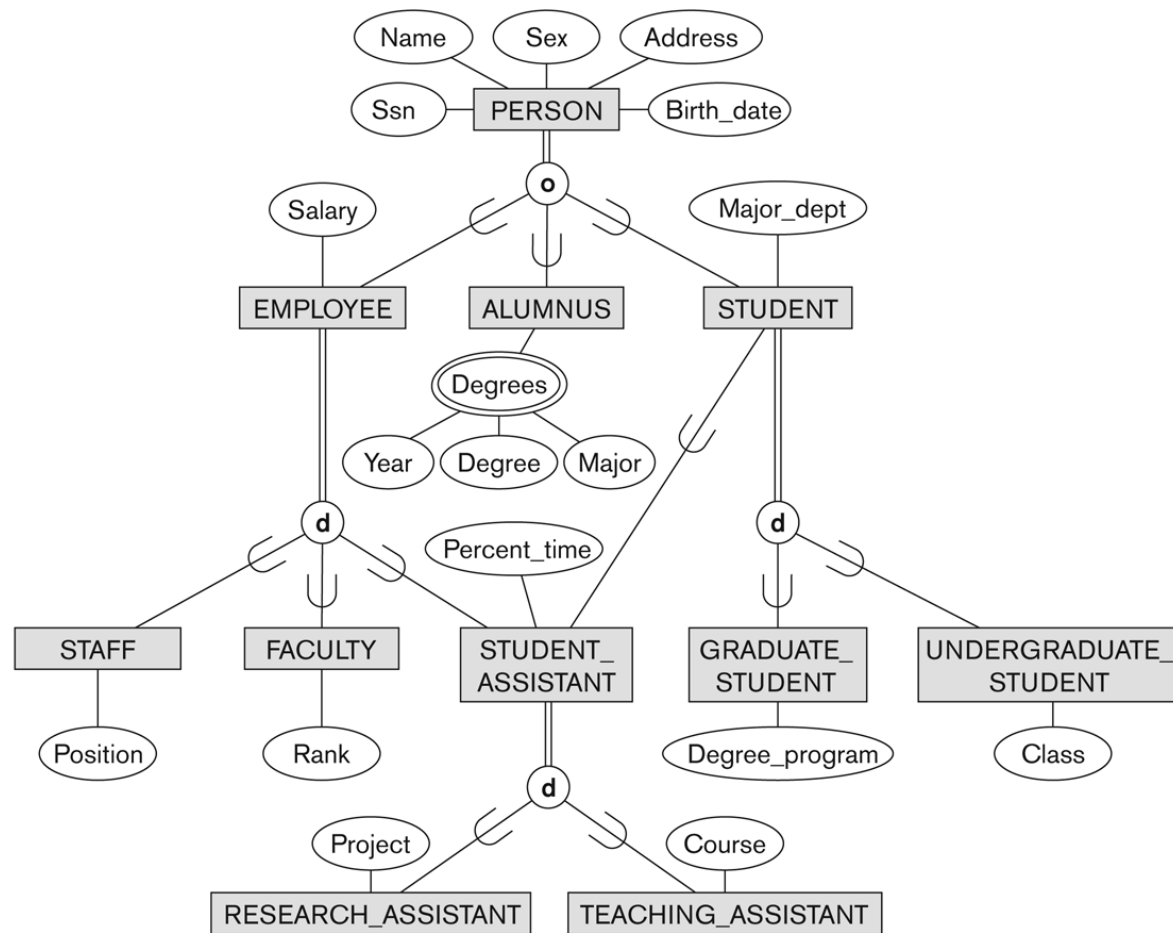


Figure 4.7

A specialization lattice with multiple inheritance for a UNIVERSITY database.

Summary

- Introduced the EER model concepts
 - Class/subclass relationships
 - Specialization and generalization
 - Inheritance
- Constraints on EER schemas
- These augment the basic ER model concepts introduced in Chapter 3